COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURES AND ALGORITHMS**	**SECTION:**	CPE21S4
DATE PERFORMED:	8 / 7 / 26	**DATE SUBMITTED:**	8 / 8 / 26
NAME:	Mangcucang, Rhianne L.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 5 **QUEUES**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

```
#include <iostream>
#include <queue>

void display(std::queue<std::string> q)
{
    while (!q.empty())
    {
        std::cout << "<- " << q.front();
        q.pop();
    }

    std::cout << "\n";
}
```

The display function is built to show all the names in the queue without destroying the original list. It works by taking a temporary copy of the queue called q, meaning anything deleted inside this function will not affect the actual data in main. It then uses a while loop that keeps running as long as there is still someone in the queue. Inside the loop, it looks at the person at the front using q.front(), prints their name to the screen, and then removes them from the temporary copy using q.pop(). Removing the front person allows the next person in line to step forward so the loop can print them next. Once everyone in the copy has been printed and the queue becomes empty, the loop stops, and the function prints a simple newline \n to move the cursor down for clean formatting.

```cpp
int main()
{
    std::queue<std::string> students;

    // Insert students' names into the queue
    students.push("Rhianne");
    students.push("Elijhen");
    students.push("Jheyian");
    students.push("Ashe");

    std::cout << "The queue is: ";
    display(students);

    std::cout << "Is the queue empty? : "
              << students.empty() << "\n";

    std::cout << "Queue size() : "
              << students.size() << "\n";

    std::cout << "Queue front() : "
              << students.front() << "\n";

    std::cout << "Queue back() : "
              << students.back() << "\n";

    std::cout << "\nAfter a pop() : ";
    students.pop();
    display(students);

    students.push("Brim");

    std::cout << "\nAfter a push() : ";
    display(students);

    return 0;
}
```

The main function is where the entire program begins running. It starts by creating an empty queue named students designed to hold text names. It then adds four names—"Rhianne", "Elijhen", "Jheyian", and "Ashe"—one by one using push(), putting them in line in the exact order they were added. After printing a label, it calls the display function to show the whole line on screen. Next, it tests several basic queue features: empty() checks if the queue has no people (which outputs 0 for false), size() counts how many people are in line (which outputs 4), front() shows the first person in line ("Rhianne"), and back() shows the last person in line ("Ashe"). After that, it calls pop() to remove the first person ("Rhianne") from the front, and calls display again to show that "Elijhen" is now at the front. Finally, it uses push("Brim") to add a new person to the back of the line, calls display one last time to show the final updated queue, and finishes with return 0; to end the program smoothly.

ILO B (MAIN)

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"

int main() {

/*
    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data
    |   |   | << " back: " << back->data << "\n";
    |
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);

    std::cout << "front: " << front->data
    |   |   | << " back: " << back->data << "\n";
    |
    //display ()
    display(front);

    dequeue(&front, &back);
    std::cout << "front: " << front->data
    |   |   | << " back: " << back->data << "\n";
    |
    //display ()
    display(front);

    dequeue(&front, &back);
    std::cout << "front: " << front->data
    |   |   | << " back: " << back->data << "\n";
    |
    //display ()
    display(front);

```

The main function starts by setting up pointers named front and back to nullptr, which represents an empty linked-list queue. It then adds the number 1 to the queue using enqueue(1, &front, &back), which creates the first node and sets both front and back to point to it, printing 1 as both the front and back value. Next, it adds 2 and 3 to the queue, updating back to point to 3 while the front remains at 1, and then prints the updated front (1) and back (3) values. After printing the full queue using display(front), it removes the first item (1) using dequeue(&front, &back), which shifts the front pointer to 2. It then prints the new front (2) and back (3) values, displays the remaining queue, and repeats the same process by dequeuing 2, leaving 3 as both the front and back value before displaying the queue one final time.

```
    dequeue(&front, &back);

    //display ()
    display(front);

    dequeue(&front, &back);
*/

    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Enqueue(2);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Enqueue(3);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Enqueue(4);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Enqueue(5);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Enqueue(6);
    std::cout << "Front: " << q.Front() << " | Back: " << q.Back() << " | Size: " << q.Size() << std::endl;

    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();

    return 0;
}
```

This code tests an array-based queue named q with a fixed capacity of 6 using queueArr<int> q(6);. It adds numbers from 1 to 6 one by one using q.Enqueue(), and after each addition, it prints the current front element using q.Front(), the back element using q.Back(), and the total number of items using q.Size(). As each number is pushed into the queue, the front element stays at 1, the back element updates to the newly added number (moving from 1 up to 6), and the size increases incrementally from 1 to 6. Finally, it executes q.Dequeue() six times in a row to remove all elements from the queue in order before ending the program with return 0;.

QUEUEARR.H

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

template <typename T>
class queueArr{
    private:
        //pointer to dynamically allocated array
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        //constructor
        queueArr(size_t capacity = 10);

        bool Empty();
        size_t Size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T value);
        void Dequeue();

        //copy constructor:
        queueArr(const queueArr& other);

        // copy assignment operator
        queueArr& operator=(const queueArr& other);

        // destructor
        ~queueArr();
};
```

This code creates a blueprint named queueArr for a custom queue that uses a dynamic array to hold elements of any data type. In the private section, it sets up five essential variables to manage the queue behind the scenes: q_array points to the array in memory where items are stored, q_capacity keeps track of the total slots available, q_size counts how many slots are currently used, while q_front and q_back keep track of the index positions for where the first item starts and where the last item ends.

In the public section, it provides all the tools you need to use the queue in your main program. It begins with a constructor that creates a new queue with a default capacity of 10 items if you do not specify a size. It includes simple checker functions like Empty() to see if the queue is clear, Size() to check the item count, and Clear() to reset the queue. To look at data without removing it, Front() lets you peek at the first item in line while Back() shows the last item. To change the queue, Enqueue(T value) adds a new item to the end, and Dequeue() removes the item at the front. Lastly, it includes essential memory management functions—a copy constructor, a copy assignment operator, and a destructor ~queueArr()—which work together to safely copy queue objects and delete dynamically allocated memory when the queue is no longer needed.

```
39   //constructor
40   template<typename T>
41   queueArr<T>::queueArr(size_t capacity){
42       q_capacity = capacity;
43       q_size = 0;
44       q_front = 0;
45       q_back = -1;
46       q_array = new T[q_capacity];
47   }
48
49   template <typename T>
50   bool queueArr<T>::Empty(){
51       return q_size == 0;
52   }
53
54   template<typename T>
55   size_t queueArr<T>::Size(){
56       return q_size;
57   }
58
59   template<typename T>
60   void queueArr<T>::Clear(){
61       q_size = 0;
62       q_front = 0;
63       q_back = -1;
64   }
```

This code shows the implementation of basic functions for the queueArr class. First, the constructor sets up a brand-new queue by taking a given capacity and saving it into q_capacity. It resets the queue by setting q_size to 0, sets the starting index q_front to 0, and sets q_back to -1 so that the very first item added will go into index 0. Finally, it uses the new keyword to create a actual dynamic array in memory with the requested capacity.

Below the constructor, three helper functions handle simple status checks and resets. The Empty() function checks if q_size == 0, returning true if the queue has no items and false if it contains data. The Size() function simply returns q_size to tell you how many items are currently inside the queue. Lastly, the Clear() function wipes the queue clean without deleting the array by resetting q_size back to 0, q_front back to 0, and q_back back to -1, making the array ready to store new data from scratch.

```
66  template<typename T>
67  T queueArr<T>::Front(){
68      //what will happen if the queue is already empty?
69      if(Empty()){
70          std::cout<< "The Queue is Empty.\n";
71          return T{};
72      }
73
74      return q_array[q_front];
75  }
76
77  //back
78  template <typename T>
79  T queueArr<T>::Back(){
80      if(Empty()){
81          std::cout<< "The Queue is Empty.\n";
82          return T{};
83      }
84
85      return q_array[q_back];
86  }
87
88  template <typename T>
89  void queueArr<T>::Enqueue(T value){
90      //check if the queue is full
91      if(q_size == q_capacity){
92          std::cout<< "Queue is full\n";
93          return;
94      }
```

This code shows three methods for managing data in the queueArr class. First, the Front() function lets you peek at the item at the start of the queue. It first checks if the queue is empty using Empty(), and if so, it prints "The Queue is Empty.\n" and returns a default blank value (T{}). If the queue contains items, it returns the value stored at q_array[q_front].

Second, the Back() function works almost identically to Front(), but it lets you peek at the item at the end of the line. It checks if(Empty()), prints an error message, and returns a default value if there are no items. If the queue has data, it returns the item stored at q_array[q_back].
Third, the Enqueue(T value) function begins the process of adding a new item to the back of the queue. Before adding anything, it checks if the queue is full by comparing q_size == q_capacity. If the queue has reached its max capacity, it prints "Queue is full\n" and immediately stops execution using return to prevent overflow.

```
96      // compute for the q_back for circular (use the modulo operator)
97      q_back = (q_back+1) % q_capacity;
98      // assign the q_array with the updated q_back to the value
99      q_array[q_back] = value;
100     // increment the q_size
101     q_size++;
102     return;
103 }
104
105 template <typename T>
106 void queueArr<T>::Dequeue(){
107     //check if it is empty
108     if(Empty()){
109         std::cout<< "the queue is empty\n ";
110         return;
111     }
112     //move q_front circularly
113     q_front = (q_front + 1) % q_capacity;
114
115     // decrement the q_size
116     q_size--;
117
118     //reset index when queue becomes empty
119     if(Empty()){
120         Clear();
121         return;
122     }
```

The Dequeue() function handles removing the item at the front of the queue after it has been processed. It begins by checking if the queue is empty using if(Empty()), and if so, it prints "the queue is empty\n " and immediately stops execution using return. If there are items in the queue, it advances the front pointer to the next item using q_front = (q_front + 1) % q_capacity, where the modulo operator ensures the pointer safely wraps back to index 0 if it reaches the end of the array. It then decreases the total item count by calling q_size--. Finally, it checks if(Empty()) again to see if that was the last remaining item; if the queue has now become completely empty, it calls Clear() to reset the internal pointers back to their starting positions and exits early.

```cpp
// copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    // allocate a new array for deep copy
    q_array = new T[q_capacity];

    // copy elements to the new array
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

// copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other){
    // check for self-assignment
    if (this == &other) {
        return *this;
    }

    // free existing dynamic memory
    delete[] q_array;

    // copy member variables
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
```

The copy constructor handles creating a brand new queue that is an exact duplicate of an existing one. It starts by copying the basic tracking variables like q_capacity, q_size, q_front, and q_back from the other queue into the new one. To ensure the new queue has its own independent storage space (called a "deep copy"), it allocates a fresh array in memory using q_array = new T[q_capacity]. Finally, it uses a for loop to go through the original queue's array and copy every single item over to the new array, ensuring that modifying one queue later won't accidentally change the other.

The copy assignment operator handles updating an already existing queue to match another one (like when you use the equals sign: queue1 = queue2). First, it checks for "self-assignment" using if (this == &other) to make sure you aren't trying to copy a queue into itself; if they are the same, it safely stops and returns the current object. Next, it deletes the existing dynamic memory using delete[] q_array to clear out the old data and prevent memory leaks. After clearing the old data, it copies over the basic tracking variables from the other queue, and then proceeds to allocate new memory and copy the data items over just like the copy constructor does.

```
160     // allocate new array and copy data
161     q_array = new T[q_capacity];
162     for (size_t i = 0; i < q_capacity; i++) {
163         q_array[i] = other.q_array[i];
164     }
165
166     return *this;
167 }
168
169 template <typename T>
170 queueArr<T>::~queueArr(){
171     delete[] q_array;
172 }
173
174 #endif // QUEUEARR_H
```

Continuing the copy assignment operator, lines 161 to 164 allocate a new dynamic array using q_array = new T[q_capacity] to fit the updated capacity, followed by a for loop that copies each element individually from other.q_array into the newly created space. Once all elements are copied, line 166 executes return *this;, which returns a reference to the current object so that assignment chaining (such as a = b = c;) works smoothly. Finally, lines 169 to 172 define the class destructor ~queueArr(). Its job is cleanup: when a queueArr object goes out of scope or is deleted, delete[] q_array; automatically deallocates the dynamic array memory created on the heap, ensuring the program frees up memory properly and prevents memory leaks.

OUTPUT

```
Front: 1 | Back: 1 | Size: 1
Front: 1 | Back: 2 | Size: 2
Front: 1 | Back: 3 | Size: 3
Front: 1 | Back: 4 | Size: 4
Front: 1 | Back: 5 | Size: 5
Front: 1 | Back: 6 | Size: 6
the queue is empty
```

ILOB (MAIN)

```cpp
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data
              << " back: " << back->data << "\n";

    enqueue(2, &front, &back);
    enqueue(3, &front, &back);

    std::cout << "front: " << front->data
              << " back: " << back->data << "\n";

    //display ()
    display(front);

    dequeue(&front, &back);
    std::cout << "front: " << front->data
              << " back: " << back->data << "\n";

    //display ()
    display(front);

    dequeue(&front, &back);
    std::cout << "front: " << front->data
              << " back: " << back->data << "\n";

    //display ()
    display(front);

    dequeue(&front, &back);

    //display ()
    display(front);

    dequeue(&front, &back);


    return 0;
}
```

This main program demonstrates how a linked-list-based queue operates step-by-step. It begins on lines 6 and 7 by declaring two pointers, front and back, initializing both to nullptr to represent an empty queue. Line 9 calls

enqueue(1, &front, &back) to add the integer 1, making both front and back point to this single node, which lines 10 to 11 print out to show that front: 1 and back: 1.

Next, lines 13 and 14 call enqueue two more times to add 2 and 3 to the back of the queue. Lines 16 to 17 then print the updated queue status (front: 1 and back: 3), followed by line 20 calling display(front) to print all elements in the queue from front to back.

Lines 22 to 36 demonstrate removing elements using dequeue(&front, &back). Line 22 removes the first element (1), leaving 2 as the new front (front: 2, back: 3), which is printed along with the updated contents via display(front). Line 29 removes 2, leaving only 3 in the queue (front: 3, back: 3), which is again printed using display(front). Line 36 removes the last element (3), leaving the queue completely empty.

Finally, lines 39 to 41 display the now-empty queue and attempt one last dequeue operation on an empty queue to demonstrate how the function gracefully handles underflow. Line 44 completes the program with return 0;, signaling successful execution.

QUEUELL.H

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode<T>* next;
};

//creating the new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
    //create a new node:
    qNode<T>* newNode = new_node<T>(newData);

    //inserting into an empty queue

    //how do we know that the queue is empty?
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        //assign the frontPtr to the newNode
        (*frontPtr) = newNode;

        //assign the backPtr to the newNode
        (*backPtr) = newNode;

        return;
```

This section sets up the basic building blocks and handles adding new data to the queue. It starts by creating a qNode structure, which holds two things: a data value (data) and a pointer (next) that points to the next item in line. Next, the new_node function acts as a helper that creates a single new node in memory, places the newData inside it, sets its next pointer to NULL, and hands it back. The enqueue function uses this helper to add a new item to the back of the queue. It checks if the queue is empty by testing if both *frontPtr and *backPtr are equal to nullptr. If the queue is completely empty, it sets both *frontPtr and *backPtr to point directly to this newly created node.

```
    }

    //point the back next to the newNode:
    (*backPtr)->next = newNode;

    //the newnode will be the new backPtr
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //how to detemerin if the queue is empty?
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout << "the queue is empty. Nothing to dequeue\n";
        return;
    }

    //creating a temporary pointer
    qNode<T>* tempPtr = nullptr;

    //store the frontPtr to the tempPtr
    tempPtr = (*frontPtr);

    //check if queue has only 1 node left
    if ((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;

        delete tempPtr;
        return;
    }
```

This image shows how enqueue handles a queue that already has items, followed by the start of the dequeue function. If the queue is not empty, enqueue connects the current last node to the new node using (*backPtr)->next = newNode and then updates *backPtr so that it points to the new node at the very end. The dequeue function handles removing the item at the front of the queue. It first checks if the queue is empty using if ((*frontPtr) == nullptr && (*backPtr) == nullptr), printing an error message and stopping if there is nothing to remove. If items exist, it creates a temporary pointer called tempPtr to keep track of the current front node. If there is only one node left in the queue—checked using if ((*frontPtr) == (*backPtr))—it resets both *frontPtr and *backPtr to nullptr and deletes tempPtr from memory.

```
    //update the frontPtr
    (*frontPtr) = (*frontPtr)->next;

    //point the temp -> to null
    tempPtr->next = nullptr;

    delete tempPtr;
}

//display all items in the queue: check the traverse in singlyLL
template <typename T>
void display(qNode<T>* frontPtr) {

    if (frontPtr == nullptr) {
        std::cout << "the queue is empty.\n";
        return;
    }

    qNode<T>* tempPtr = frontPtr;

    while (tempPtr != nullptr) {
        std::cout << tempPtr->data << " ";
        tempPtr = tempPtr->next;
    }

    std::cout << "\n";
}

#endif
```

This final image shows how dequeue finishes removing an item when there are multiple nodes, as well as the complete display function. If the queue has more than one node, dequeue shifts *frontPtr forward to the second node using (*frontPtr) = (*frontPtr)->next, disconnects the old front node by setting tempPtr->next = nullptr, and deletes tempPtr to free up memory. The display function prints out all the items currently in the queue from front to back. It starts by checking if frontPtr == nullptr, printing "the queue is empty.\n" if there is no data to show. If items exist, it uses a temporary pointer (tempPtr) and a while (tempPtr != nullptr) loop to print each tempPtr->data with a space and move on to the next node using tempPtr = tempPtr->next until it reaches the end of the line.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

MAIN

```cpp
#include <iostream>
#include "printer.h"

int main() {
    Printer officePrinter;

    std::cout << "=== Incoming Print Requests ===\n";
    officePrinter.addJob(101, "Marco", 4);
    officePrinter.addJob(102, "Sophia", 15);
    officePrinter.addJob(103, "Liam", 8);
    officePrinter.addJob(104, "Chloe", 5);

    std::cout << "\n=== Executing Print Queue (FIFO) ===\n\n";
    officePrinter.processAllJobs();

    std::cout << "\n=== Second Batch of Print Jobs ===\n";
    officePrinter.addJob(105, "Ethan", 10);
    officePrinter.addJob(106, "Mia", 3);

    std::cout << "\n";
    officePrinter.processAllJobs();

    return 0;
}
```

H. FILE

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <string>

class Job {
public:
    int id;
    std::string userName;
    int pages;

    Job(int jobId = 0, std::string name = "", int numPages = 0) {
        id = jobId;
        userName = name;
        pages = numPages;
    }
};

struct Node {
    Job data;
    Node* next;

    Node(Job job) {
        data = job;
        next = nullptr;
    }
};

class Printer {
private:
    Node* front;
    Node* rear;
    int count;

public:
    Printer() {
```

```
37    Printer() {
38        front = nullptr;
39        rear = nullptr;
40        count = 0;
41    }
42
43    ~Printer() {
44        while (front != nullptr) {
45            Node* temp = front;
46            front = front->next;
47            delete temp;
48        }
49    }
50
51    void addJob(int id, std::string userName, int pages) {
52        Job newJob(id, userName, pages);
53        Node* newNode = new Node(newJob);
54
55        if (rear == nullptr) {
56            front = rear = newNode;
57        } else {
58            rear->next = newNode;
59            rear = newNode;
60        }
61        count++;
62        std::cout << "Job #" << id << " submitted by " << userName << " (" << pages << " pages) has been added to the queue.\n";
63    }
64
65    void processAllJobs() {
66        if (front == nullptr) {
67            std::cout << "No jobs to process.\n";
68            return;
69        }
70
71        std::cout << "Printer is now processing " << count << " job(s)...\n";
```

```
71        std::cout << "Printer is now processing " << count << " job(s)...\n";
72
73        while (front != nullptr) {
74            Node* temp = front;
75            std::cout << "Printing Job #" << temp->data.id
76                      << " for " << temp->data.userName
77                      << " - " << temp->data.pages << " page(s)... Done.\n";
78
79            front = front->next;
80            if (front == nullptr) {
81                rear = nullptr;
82            }
83            delete temp;
84        }
85
86        count = 0;
87        std::cout << "All jobs have been processed. Printer is idle.\n";
88    }
89    };
90
91    #endif
```

OUTPUT

```
=== Incoming Print Requests ===
Job #101 submitted by Marco (4 pages) has been added to the queue.
Job #102 submitted by Sophia (15 pages) has been added to the queue.
Job #103 submitted by Liam (8 pages) has been added to the queue.
Job #104 submitted by Chloe (5 pages) has been added to the queue.

=== Executing Print Queue (FIFO) ===

Printer is now processing 4 job(s)...
Printing Job #101 for Marco - 4 page(s)... Done.
Printing Job #102 for Sophia - 15 page(s)... Done.
Printing Job #103 for Liam - 8 page(s)... Done.
Printing Job #104 for Chloe - 5 page(s)... Done.
All jobs have been processed. Printer is idle.

=== Second Batch of Print Jobs ===
Job #105 submitted by Ethan (10 pages) has been added to the queue.
Job #106 submitted by Mia (3 pages) has been added to the queue.

Printer is now processing 2 job(s)...
Printing Job #105 for Ethan - 10 page(s)... Done.
Printing Job #106 for Mia - 3 page(s)... Done.
All jobs have been processed. Printer is idle.
```

The program output shows how a simple First-In, First-Out (FIFO) queue works for a shared office printer. When the first group of print requests was added, every job went straight to the back of the line. The printer then processed Marco's job first simply because he submitted it ahead of everyone else, followed by Sophia, Liam, and Chloe in exact arrival order. Once all four jobs finished, the system cleared out the queue and went idle. When the next batch from Ethan and Mia came in, the printer started up again and handled them the exact same way. This proves that our linked list handles dynamic print requests smoothly without mixing up the order or dropping any data.

CONCLUSION

In this activity, I learned how to build a custom Queue data structure using a Linked List in C++ without relying on built-in libraries. It helped me understand how pointers actually work in real scenarios, specifically how we append nodes to the back when adding jobs and delete nodes from the front once they are processed. It made it really easy to see how shared resources like office printers manage multiple requests fairly.

Dividing the project into two files—printer.h for the classes and main.cpp for running the simulation—kept everything clean and easy to troubleshoot. Using dynamic memory allocation with new and delete was very effective here because memory was only used when there were active jobs in line, and it got freed up right away as soon as each job finished printing.

Running multiple batches of print jobs showed why linked lists are great for this kind of setup. Unlike arrays, which have a fixed maximum size and force you to shift elements around every time someone leaves the line, the linked list structure lets jobs enter and leave the queue smoothly without worrying about running out of pre-allocated array space.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.